5 RoutingBlocks: An open-source Python package for Vehicle Routing Problems with Intermediate Stops

This chapter is based on an article published as:

Klein P. S., Schiffer M. (2024) RoutingBlocks: An Open-Source Python Package for Vehicle Routing Problems with Intermediate Stops. *INFORMS Journal on Computing* (Articles in Advance).

Patrick S. Klein
School of Management, Technical University of Munich, Munich, Germany,
patrick.sean.klein@tum.de

Maximilian Schiffer
School of Management & Munich Data Science Institute, Technical University of Munich, Munich, Germany,
schiffer@tum.de

We introduce `RoutingBlocks`, a versatile open-source Python package designed to simplify the development of algorithms for vehicle routing problems with intermediate stops (VRPIS). The package offers a variety of modular algorithmic components and optimized data structures, crafted specifically to address key challenges of VRPIS, such as a lack of efficient exact move evaluations and difficult station visit decisions. By employing a unified solution and instance representation that abstracts problem-specific behavior, e.g., constraint checking, move evaluation, and cost computation, into well-defined interfaces, `RoutingBlocks` maintains a clear separation between algorithmic components and specific problem configurations, thus allowing to apply the same algorithm to a variety of problem settings. Leveraging an efficient C++ implementation for performance-critical core elements, such as move evaluation and local search operators, `RoutingBlocks` combines the high performance of C++ with the user-friendliness and adaptability of Python, thereby streamlining the development of effective metaheuristic algorithms. As a result, researchers using `RoutingBlocks` can focus on their algorithms' core features, allocating more resources to innovation and advancement in the VRPIS domain.

Key words: Vehicle routing, metaheuristic algorithms, Python, open-source software

5.1. Introduction

Vehicle routing problems with intermediate stops (VRPIS) have become ubiquituous in transport optimization when focusing on recently evolving planning problems, e.g., the deployment of electric vehicles or innovative urban logitics concepts where city freighters perform last-mile delivery, replenishing freight at micro depots (cf. Schiffer et al. 2019). In VRPIS, vehicles perform *intermediate stops* at *intraroute facilities* to replenish a resource, e.g., energy or freight, to keep a vehicle operational while fulfilling its main task. Here, an intraroute facility differs from a conventional depot as it is on the same echelon as the customers, e.g., a charging station or a micro depot.

During the last decade, we have seen steep progress on developing state-of-the-art open source meta-heuristic and exact frameworks for classical vehicle routing problem (VRP) vari-

ants such as the capacitated vehicle routing problem (CVRP) and the vehicle routing problem with time windows (VRPTW) (see, e.g., Accorsi and Vigo 2021, Vidal 2022). Contrarily, the progress on state-of-the-art algorithms for VRPIS variants appears to emerge rather slowly. In the case of exact algorithms, this is likely caused by a lack of VRPIS-specific algorithms (cf. Schiffer et al. 2019). For metaheuristic algorithms, this can partially be attributed to missing state-of-the-art open-source implementations tailored to VRPIS.

In fact, extending metaheuristics or open source libraries designed for classical VRP variants to VRPIS is anything but straightforward for several reasons: first, classical algorithms rely heavily on local search, which is insufficient for accommodating intermediate stops as it focuses on cost-minimization. Indeed, inserting intermediate stops often causes a detour which is orthogonal to the goal of local search. Second, existing algorithms rely on efficient, exact evaluation of local search moves in amortized constant time. However, for many well-known VRPIS, e.g., electric vehicle routing problems (EVRPs), no such evaluation functions exist such that move evaluation remains expensive or approximate (Montoya et al. 2017, Schiffer and Walther 2018, Erdelić and Carić 2022). Third, different VRPIS variants often show unique problem characteristics, e.g. w.r.t. replenishment behavior (quantitiy-dependent vs. quantity-independent), replenishment and resource trade-offs, and synchronization constraints (cf., Schiffer et al. 2019), that require specialized algorithmic components. Accordingly, designing a library that is amendable to a broad range of VRPIS is a challenge in itself. In fact, even for mainstream VRPs, existing libraries (Groer 2009, Rasku and Kwon 2023) and open sourced metaheuristics (Vidal 2022) struggle with this generalization task and surpass it by either modeling the union of all problem-specific characteristics[1] (Groer 2009, Vidal 2022), or by not providing a generalized interface for, e.g., cost evaluation and constraint checking (Rasku and Kwon 2023). Both of these approaches have their respective drawbacks: considering the union of all problem-specific characteristics makes the implementation more challenging and requires additional cognitive effort on the side of the library user. Not providing a central interface on the other hand limits the composeability and interoperability of the implemented algorithms, which often use different solution representations and evaluation procedures.

Against this background, we provide an open-source Python package—`RoutingBlocks`—that eases the design and implementation of algorithms for VRPIS. Here, `RoutingBlocks`

[1] using dummy values for problems that only require a subset of these

specifically focusses on metaheuristics to best align with existing research in the area and remain amendable to broad range of applications, e.g., dynamic settings. `RoutingBlocks` breaks with the design of existing open-source VRP packages to account for the peculiarities highlighted above, and, instead of implementing a single, monolithic algorithm, provides a library of modular algorithmic components and efficient data structures. We specifically tailor these to handle some of the fundamental challenges that arise when solving VRPIS. The modular design of our package significantly eases the development of metaheuristics for VRPIS, allowing researchers to focus on implementing the primary algorithmic flow by composing the provided components, with an easy option to add specialized supplementary components if necessary. Ultimately, researchers can focus on the core aspects of their algorithms when utilizing our package, using more of their resources to facilitate innovation and progress in the field of VRPIS.

The remainder of this paper is structured as follows. Section 5.2 gives a general definition of the class of VRPIS considered when developing the `RoutingBlocks` package. Section 5.3 then provides an overview on the architecture and design of `RoutingBlocks` and briefly reviews the package's algorithmic components. Section 5.4 provides information on obtaining the package, before Section 5.5 benchmarks an algorithm implemented using `RoutingBlocks` against state-of-the-art monolithic designs. Section 5.6 concludes the paper. We note that this paper details the design of Routingblocks and supplements the documentation[2], which provides a hands-on introduction to the package in addition to several example implementations.

5.2. Vehicle routing problems with intermediate stops

VRPIS expand upon classic VRPs by considering one or multiple additional resources consumed when traveling along arcs and servicing customers. Unlike resources in classical VRPs, e.g., time, these resources do not increase or decrease monotonously. Instead, they can be replenished at intermediate facilities, typically by facing a trade-off between different resources. An example of a VRPIS is the electric vehicle routing problem with time windows (EVRP-TW), where a fleet of electric vehicles with limited battery capacity must serve a set of customers within specified time windows. In this case, the consumed additional resource

[2] `https://routingblocks.readthedocs.io/`

is the battery's state of charge, which can be replenished by charging at designated charging stations, consuming time in the process.

Formally, VRPIS are defined on directed complete graphs, which consist of vertices representing locations that vehicles can visit during their routes, specifically depots, customers, and intermediate stops. Arcs connect these vertices, and both arcs and vertices have resource consumption functions that determine the consumption of each considered resource incurred by traversing the arc or visiting the vertex. These resource consumption functions can be arbitrary and are often non-linear, e.g., in some EVRP variants. For a formal definition of VRPIS, we refer the interested reader to Schiffer et al. (2017).

5.3. The `RoutingBlocks` package

`RoutingBlocks` is an open-source Python package that eases the implementation of algorithms for VRPIS by providing a diverse set of modular algorithmic components and efficient data structures (see Figure 1), designed specifically to tackle some of the fundamental challenges of VRPIS, e.g., inexact move evaluation and difficult decisions around embedding visits to intraroute replenishment facilities. These components serve as building blocks for problem-specific algorithms and include a local search solver (Sections 5.3.1 and 5.3.2), an exact labelling algorithm (Section 5.3.3), components for metaheuristic algorithms (Section 5.3.4), and auxiliary algorithms and data structures, e.g., move caches (Section 5.3.5). Each of these can be extended through well-defined interfaces to support a wide range of algorithmic designs.

As illustrated in Figure 1, `RoutingBlocks` consists of three modules: *algorithms and data structures*, *core components*, and *problem-specific interfaces*. Algorithms and data structures implement components commonly used in metaheuristics for VRPIS and utilize a common solution and instance representation provided by the core components. These abstract problem-specific behavior, such as constraint checking, move evaluation, and cost computation, such that algorithms remain decoupled from specific problem settings. Users can incorporate problem-specific behavior by implementing the provided interfaces.

`RoutingBlocks` builds on an efficient C++-based implementation of runtime-critical core components. This integration of C++ and Python combines the high performance of C++ with the ease of use and flexibility of Python, allowing rapid development of efficient metaheuristic

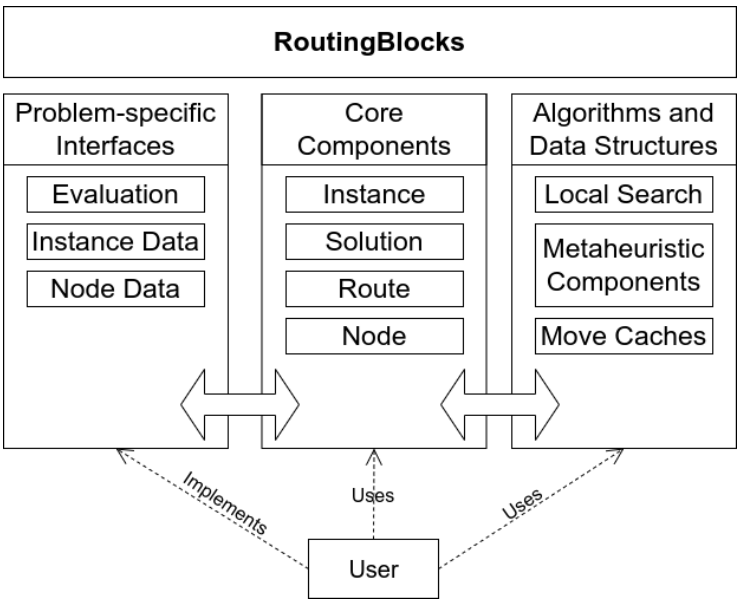


Figure 1 Architecture of the `RoutingBlocks` package.

algorithms. Users can thus implement the core logic of their algorithms in Python, potentially capitalizing on existing high-quality implementations of other complex algorithmic components (see, e.g., Kullman et al. (2021)) and leveraging robust tooling (Peng and Murray 2022, Mendoza et al. 2014). Beyond this, `RoutingBlocks` supports using any of the provided components straight from C++ as well. Accordingly, our package supports customization of the provided algorithmic components using C++, Python, or a combination of the two.

In what follows, we briefly review the design of our core components, i.e., the instance and solution representations, as well as the algorithms and data structures our package provides. We refer to `https://github.com/tumBAIS/RoutingBlocks` for comprehensive documentation of the individual components and an example that illustrates how to compose these to a full algorithm.

5.3.1. Core components

Core components capture the fundamental concepts shared by all VRPIS, such as instance graphs, routes, and solutions. By providing abstract interfaces, they effectively conceal problem-specific data, e.g., customer time windows, and behavior, e.g., cost evaluation procedures, from the provided data structures and algorithms.

5.3.1. Instance Graph Similar to regular VRPs, VRPIS are almost exclusively modeled on graphs, which we represent using the instance class. This class contains a set of vertices,

that correspond to locations vehicles can visit during their routes. We divide vertices into three groups: depots, customers and intermediate stops. The class also contains a set of arcs, represented as a dense two-dimensional matrix, where each row corresponds to the set of arcs that originate from a particular vertex.

The instance class assumes only minimal knowledge about concrete vertices and arcs. Specifically, vertices carry only an identifier and a type (i.e., station, depot, or customer), and arcs do not carry any information at all. Instead, these provide type-erased slots for user-defined problem-specific data, e.g., time windows, demand, and distance. Problem-specific details thus remain opaque to the instance but are retrievable by specialized algorithms.

5.3.2. Solution Representation Our framework provides a rich, intuitive solution representation comprising *solution*, *route*, and *node* classes, organized hierarchically: a solution comprises a set of routes, which each manage a sequence of nodes. Here, nodes represent visits to vertices and carry data used for cost calculation, constraint checking, and efficient move evaluation (cf. Section 5.3.2). Clearly, the structure of this data and how it is utilized remains highly problem-specific and thus has to remain opaque to route and solution classes. These operations are instead implemented in a problem-specific *evaluation* class (cf. Section 5.3.3). This reduces the responsibility of route and solution classes to managing the sequences of nodes and set of routes, respectively. Specifically, they provide interfaces to insert, remove, or exchange arbitrary segments of nodes, guaranteeing that information on feasibility, costs, and the data associated with partial routes, i.e., what is carried in node classes, is maintained at all times. This simplifies the implementation of specific algorithms, and further allows to abstract from problem-specific characteristics by default.

Similar to Vidal (2022), we assign monotonic *timestamps* to routes, which can be utilized to implement caching strategies. Specifically, we guarantee that routes with matching timestamps are equal with respect to the sequence of nodes visited. We achieve this with a simple global counter: each time a route is modified, we set it's timestamp to the current value of the counter, which is incremented subsequently.

5.3.3. Evaluation The evaluation class provides interfaces to i) assess the cost of a (partial) route, ii) verify the feasibility of a (partial) route, and iii) maintain the information contained within node structures. It's interfaces are implemented by the user, such that

the class has access to data that is opaque to instance, solution, route, and node classes. The design of this interface follows the concept of propagation functions and route segment concatenation introduced in Vidal et al. (2014), which we further detail in Section 5.3.2.

5.3.2. Data structures and algorithms

5.3.1. Local Search Local search is a core component of almost every state-of-the-art metaheuristic designed for VRPs. The main idea of this algorithm is to iteratively improve an initial solution by making small, localized modifications, so called *moves*, to the current solution until reaching a local optimum, i.e., a state where none of the defined moves further improve the solution. This seemingly simple concept allows for a lot of customization, such that the specific design of the local search procedure varies significantly between algorithms. Common design decisions include i) the selection of neighborhood operators, which define the considered moves, ii) pivoting rules, which dictate a strategy for selecting moves, e.g., best and first improvement, and iii) speed-up techniques such as granular neighborhoods.

`RoutingBlocks` provides a local search component that supports a variety of local search designs. Here, we deviate from generator-arc based move definitions (see, e.g., Vidal 2022). Specifically, in generator-arc based designs, each move is uniquely defined by a single arc. This allows to define operators as single, stateless functions that take an arc to be evaluated as input argument, thus allowing for a clear separation of concerns: operators simply evaluate or apply a move, and a local search solver steers the search procedure by controlling which arcs are passed to the respective operators. Unfortunately, this generator-arc based design is not well-suited for VRPIS, where widely used operators, e.g., the *station-in* operator (Schneider et al. 2014) that attempts to insert detours to intermediate replenishment facilities into an existing route, may assess several moves per arc, and arcs can appear an arbitrary number of times within the same solution. We achieve similar benefits as generator-arc based designs by treating operators as iterators over the neighborhood of a solution. Specifically, on each invocation, an operator generates the (next) improving move, returning control to the callee. These moves remain opaque to the local search solver and simply provide apply and evaluation interfaces. This provides the flexibility required to support arbitrary operators while still allowing the local search solver to control the search. Accordingly, implementing design decisions such as pivoting rules and neighborhood composition strategies (e.g.,

variable neighborhood descent) in `RoutingBlocks` does not require implementation on an operator-to-operator basis.

`RoutingBlocks` provides well defined interfaces for general moves and operators, which can be implemented in Python or C++. It additionally provides specializations of general operator and move interfaces to support generator-arc based implementations, such that porting existing implementations of local search operators to `RoutingBlocks` is straightforward. `RoutingBlocks` further implements a set of well-known operators commonly used in VRPIS, e.g., various swap- and relocation-based neighborhoods (*n-m-exchange*), *TwoOpt**, and *Station-In/Out*, out of the box.

5.3.2. Move evaluation Computationally efficient evaluation of moves is crucial to establish the performance of local search based metaheuristics. Here, many algorithms rely on the observation that any edge- or node-exchange based move can be expressed as a concatenation of route sub-sequences (Kindervater and Savelsbergh 2003). Hence, an evaluation function that takes a list of node sequences as arguments can leverage pre-computed information on route sub-sequences to achieve computationally efficient move evaluation. For instance, removing node x_i from route $[x_1, \dots, x_{i-1}, x_i, x_{i+1}, \dots, x_n]$ can be expressed as concatenating routes $[x_1, \dots, x_{i-1}]$ and $[x_{i+1}, \dots, x_n]$, potentially making use of the *forward* information stored in x_{i-1} and the *backward* information stored in x_{i+1}. This idea has been generalized in Vidal et al. (2014). `RoutingBlocks`'s interface follows their design and exposes five functions to be implemented by the user: i) `initialize_forward(`x`)` and `initialize_backward(`x`)`, which initialize forward and backward information stored on node x, respectively, ii) `propagate_forward(`$[\dots, x_i]$`, `y`)` and `propagate_backward(`y`, `$[x_j, \dots]$`)`, which extend the forward and backward information stored for routes $[\dots, x_i]$ and $[x_j, \dots]$ to $[\dots, x_i, y]$ and $[y, x_j, \dots]$ for some node y, respectively, and iii) `evaluate(`$\sigma_1, \dots, \sigma_n$`)`, which evaluates the cost of concatenating an arbitrary number of route segments σ.

A major challenge in many VRPIS, e.g., (non-linear) EVRP-TW, is that no efficient *exact* move evaluation functions are known. A common approach to dealing with this issue is to combine efficient approximate concatenation operators with exact move evaluation functions (Montoya et al. 2017, Froger et al. 2022, Erdelić and Carić 2022). Here, the approximate operator ideally provides an upper bound on the improvement obtained from applying a move.

This allows to eagerly discard non-improving moves, such that the potentially expensive exact move evaluation function needs to be applied to only a subset of potential moves.

`RoutingBlocks`'s local search solver supports this approach by default and allows to verify improving moves using a second evaluation function. In general, any component provided by `RoutingBlocks` allows to switch evaluation implementations at any point in time.

5.3.3. Labeling algorithm Many VRPIS rely on exact algorithms to embed detours to intraroute facilities at optimal locations (Schiffer et al. 2017, Erdelić and Carić 2022). A specific approach that was proven to work well in practice models this detour embedding problem as a resource-constrained shortest path problem (CSP) on an auxilliary graph, solved for each route in the solution independently. This graph comprises a vertex for each non-station node in the route, and allows detours to these by inserting copies of each intraroute facility between two consecutive non-station vertices. We refer to Schiffer and Walther (2018) for a comprehensive description of this procedure.

`RoutingBlocks` provides an implementation of this algorithm that takes care of label management, dominance-based path pruning, graph building, and other boilerplate tasks while providing an interface to abstract problem-specific functionality, i.e., label representation, resource extension functions, dominance, and preprocessing. The design of this interface bases on the abstractions introduced in Irnich (2008).

5.3.4. Metaheuristic components Broady speaking, Metaheuristics are solution methods that guide local improvement based procedures to allow escaping from local optima. `RoutingBlocks` provides a set of data structures and boilerplate required to implement common metaheuristic algorithms. These include components that provide deterministic sources of randomness, several *destroy* operators that remove vertices from a solution, *repair operators* that insert vertices into a solution, an (adaptive) priority list to *learn* to dynamically rate operators based on past performance, and a solver that applies operators sequentially, i.e., removes and (re-)inserts a subset of vertices. Specifically, `RoutingBlocks` provides several generic destroy and repair operators known to work well for VRPIS out-of-the-box: *RandomInsertion*, *BestInsertion* (Ropke and Pisinger 2006), *WorstRemoval* (Ropke and Pisinger 2006), *RelatedRemoval* (Pisinger and Ropke 2007), *ShawRemoval* (Shaw 1997), and *StationVicinityRemoval* (Goeke and Schneider 2015). These remain extensible, e.g., the acceptance

criterion of best sequential insertion is customizable, such that N-best-insertion and best-insertion-with-blinks can be implemented straightforwardly. We provide interfaces to allow user-defined destroy and repair operators, implemented in either C++ or Python.

Our examples[3] showcase how to utilize these components to build well-known metaheuristic algorithms, e.g. *adaptive large neighborhood search (ALNS) (cf., Ropke and Pisinger 2006).*

5.3.5. Auxiliary data structures We provide several auxiliary data structures, such as insertion and removal caches, to significantly reduce the development overhead of implementing custom algorithms. Similar to the concept of static move descriptors (Zachariadis and Kiranoudis 2010, Beek et al. 2018), these caches track changes to solutions, avoiding potentially expensive re-evaluations of moves unaffected by a specific modification. Our implementation of this caching mechanism is much simpler than the one proposed in Zachariadis and Kiranoudis (2010) and Beek et al. (2018): as in Vidal (2022), we rely on route timestamps (cf. Section 5.3.2) to determine if a move has to be re-evaluated or not. The reason for this is twofold: first, maintaining complicated static move descriptor data structures can entail significant computational overhead especially in VRPIS, where the number of nodes in a solution may change dynamically. Second, this simple timestamp-based solution does not rely on any problem-specific information.

The benefit of these move caches is especially pronounced when used in algorithms implemented in Python. Here, the use of caches, which are implemented in native code, allows to bundle evaluation operations and thus minimizes the number of calls between Python and the C++ extension module.

5.4. Distribution

The `RoutingBlocks` package is available via `pip` through `PyPI` (`https://pypi.org/project/routingblocks`). It's source code, documentation, and examples can be found at `https://github.com/tumBAIS/RoutingBlocks`. The repository further provides information on contributing to the project, e.g., by reporting bugs or supplying additional algorithmic components.

[3] https://routingblocks.readthedocs.io/en/latest/examples.html

5.5. Benchmarks

We have implemented a simple metaheuristic algorithm for the electric vehicle routing problem with time windows and partial recharging (EVRP-TW-PR) using `RoutingBlocks` to show that the performance of algorithms developed with `RoutingBlocks` remains comparable to specialized, monolithic implementations, i.e., the one provided in Erdelić and Carić (2022). This particular problem setting is particularly suitable to showcase some of `RoutingBlocks'` core features, such as two-stage move evaluation, complex guiding metaheuristics, and exact subcomponents. Moreover, the algorithm proposed in Erdelić and Carić (2022) is similar to our proposed algorithm: It bases on ALNS, relies on an exact dynamic programming based station placement procedure, and uses an approximate evaluation function. Note that the purpose of this benchmark is to assess the performance of `RoutingBlocks` and not to provide new best known solutions. We thus did not conduct sensitivity analyses on the components used or tune our algorithm's hyperparameters. We refer to Erdelić and Carić (2022) for further information on the considered instances. The source code of our implementation is available on `https://github.com/tumBAIS/RoutingBlocks` and may serve as a reference implementation, showcasing how to utilize the package. Beyond this, up-to-date benchmarks that verify the performance of our core algorithms compared to pure Python-based implementations are part of our CI pipeline and available on `https://github.com/tumBAIS/RoutingBlocks`.

We conducted all of our experiments on a standard desktop computer equipped with an `Intel(R) Core(TM) i9-9900, 3.1 GHz CPU` and `16 GB` of `RAM`, running `Ubuntu 20.04` and Python `3.8.11`. We limit the total number of iterations of our ALNS algorithm to 5000, and the number of iterations without improvement to 1100. Table 1 summarizes the results of our benchmark compared to two state-of-the-art monolithic algorithms implemented in native code, i.e., `C#` and `C++` (cf. Schiffer et al. 2017, Erdelić and Carić 2022). We report detailed statistics in the Appendix.

The solution quality achieved with a `RoutingBlocks`-based implementation remains compareable to that of monolithic algorithms, maintaining a gap of 0.44% to the best known solution on average. When comparing runtime, we see that `RoutingBlocks` is 14.86% slower than a `C#`-based implementation (Erdelić and Carić 2022), and 277.93% slower than a pure `C++` implementation Schiffer et al. (2017). Despite these performance differences, the efficiency of `RoutingBlocks` is well within the bounds expected for a generic library, making it a suitable choice for prototyping metaheuristic algorithms.

n	$n^{\texttt{RB}}$	n^e	n^s	$n^{\texttt{RB}}_{\texttt{BKS}}$	$n^e_{\texttt{BKS}}$	$n^s_{\texttt{BKS}}$	$\overline{\Delta^{\texttt{RB}}_{BKS}}$ [%]	$\overline{\Delta^{e}_{BKS}}$ [%]	$\overline{\Delta^{s}_{BKS}}$ [%]	$\overline{\tau^{\texttt{RB}}}$ [s]	$\overline{\tau^{e}}$ [s]	$\overline{\tau^{s}}$ [s]
56	54	56	42	24	49	29	0.43	0.04	0.13	615.21	535.61	162.78

Abbreviations hold as follows: n - total number of instances, n° - number of instances where the respective algorithm matched the number of vehicles in the BKS, $n^\circ_{\texttt{BKS}}$ - number of instances where the respective algorithm matched the BKS, $\overline{\Delta^\circ_{BKS}}$ - average gap of instances with the same number of vehicles as the BKS expressed in percent, $\overline{\tau^\circ}$ - average runtime on instances with the same number of vehicles as the BKS in seconds. `RB` corresponds to `RoutingBlocks`, e to the algorithm of Erdelić and Carić (2022), and s to the algorithm of Schiffer et al. (2017). We caclulate percentage gaps as $\Delta^a_{\texttt{BKS}} = \frac{v^a - v_{\texttt{BKS}}}{v_{\texttt{BKS}}} \cdot 100\%$.

Table 1 Aggregated results of our benchmark against Schiffer et al. (2017) and Erdelić and Carić (2022).

5.6. Conclusion

We introduced `RoutingBlocks`, a modular Python package for the implementation of algorithms for VRPIS. `RoutingBlocks` provides a diverse set of algorithmic components and efficient data structures designed specifically to tackle some of the fundamental challenges of VRPIS, such as inexact move evaluation and complex station visit decisions. These serve as building blocks for problem-specific algorithms, streamlining the development process for algorithms addressing VRPIS and related problems. `RoutingBlocks` provides a scalable C++-based implementation as a rich Python API, combining the high performance and scalability of C++ with the ease of use and flexibility of Python. This allows the rapid development of scalable, problem-specific algorithms that can further leverage existing high-quality implementations of complex algorithms (see, e.g., Kullman et al. 2021), and leverage robust tooling (Peng and Murray 2022, Mendoza et al. 2014). In accordance with this approach, `RoutingBlocks` supports customization of the provided algorithmic components using C++, Python, or a combination of both, catering to diverse algorithmic requirements. We provide `RoutingBlocks` under a permissive open-source license, ensuring that it remains relevant and useful to the community over time. Ultimately, we hope that `RoutingBlocks` will serve as a valuable resource for anyone working on the challenging and important class of VRPIS.

Appendix A: Detailed results

Instance	`RoutingBlocks`			Erdelić and Carić (2022)			Schiffer et al. (2017)		
	σ	K	τ [s]	σ	K	τ [s]	σ	K	τ [s]
c101	1043.38	12	382.17	1043.38	12	220.20	1043.38	12	64.20
c102	1074.07	10	350.14	1058.67	10	190.20	1027.80	11	121.20
c103	973.92	10	654.90	971.19	10	248.40	973.63	10	127.80
c104	892.33	10	1070.25	884.38	10	454.20	884.48	10	135.00
c105	1064.66	10	739.17	1064.66	10	143.40	1023.52	11	81.00
c106	1091.14	10	574.49	1061.61	10	151.80	1009.26	11	58.80
c107	1088.43	10	482.79	1046.50	10	158.40	1010.91	11	54.60
c108	1055.70	10	607.18	1022.93	10	274.80	1030.48	10	105.00
c109	954.77	10	491.64	940.38	10	268.80	940.38	10	148.20
c201	629.95	4	136.11	629.95	4	49.20	629.95	4	25.20
c202	629.95	4	160.79	629.95	4	139.20	629.95	4	91.80
c203	629.95	4	207.14	629.95	4	155.40	629.95	4	199.20
c204	628.91	4	251.22	628.91	4	310.20	628.91	4	196.20
c205	629.95	4	188.13	629.95	4	63.60	629.95	4	46.80
c206	629.95	4	220.71	629.95	4	77.40	629.95	4	46.80
c207	629.95	4	233.37	629.95	4	96.60	629.95	4	72.60
c208	629.95	4	217.99	629.95	4	78.60	629.95	4	66.00
r101	1639.68	17	1815.91	1624.89	17	371.40	1618.89	18	161.40
r102	1454.92	15	1461.45	1454.53	15	441.60	1426.82	16	244.20
r103	1211.40	13	1375.46	1304.24	12	496.80	1211.99	13	358.80
r104	1056.69	11	1013.35	1051.41	11	706.20	1051.41	11	249.00
r105	1347.80	14	1234.39	1347.80	14	324.00	1347.80	14	181.20
r106	1263.13	13	1495.94	1263.13	13	510.00	1265.18	13	237.00
r107	1108.47	11	681.00	1104.51	11	635.40	1108.23	12	150.00
r108	1021.47	11	422.24	1030.44	10	696.00	1020.52	11	200.40
r109	1185.40	12	952.50	1176.69	12	502.80	1176.69	12	191.40
r110	1069.18	11	1396.62	1067.11	11	495.00	1068.87	11	226.20
r111	1073.15	11	692.80	1076.15	11	489.00	1070.97	12	241.20
r112	1001.79	11	929.92	1001.79	11	448.20	1001.79	11	204.00
r201	1255.81	3	560.94	1255.81	3	627.00	1255.81	3	88.80
r202	1051.46	3	274.08	1052.52	3	685.80	1051.46	3	151.80
r203	895.54	3	538.69	895.54	3	1144.20	895.96	3	314.40
r204	779.57	2	333.15	779.49	2	1325.40	779.49	2	428.40
r205	987.36	3	281.68	987.22	3	558.60	987.22	3	175.80
r206	925.37	3	546.18	922.08	3	701.40	922.08	3	211.20
r207	843.20	2	244.77	845.19	2	960.60	843.20	2	156.60
r208	737.30	2	275.65	736.46	2	1436.40	736.12	2	228.60
r209	871.55	3	245.37	863.17	3	668.40	863.17	3	115.20
r210	844.71	3	260.75	844.71	3	835.20	844.86	3	215.40
r211	823.82	2	548.54	825.25	2	1164.60	826.26	2	163.80
rc101	1662.16	15	590.50	1661.53	15	352.80	1648.99	16	133.80
rc102	1510.16	14	992.38	1510.16	14	461.40	1510.16	14	112.20
rc103	1361.68	12	986.65	1359.34	12	455.40	1304.10	13	181.20
rc104	1175.06	11	1118.39	1174.32	11	575.40	1175.06	11	155.40
rc105	1473.34	13	1448.29	1471.80	13	269.40	1449.53	14	179.40
rc106	1386.25	13	484.94	1391.23	13	333.60	1385.96	13	116.40
rc107	1250.80	11	524.47	1244.37	11	368.40	1248.11	12	186.00
rc108	1154.14	11	565.44	1154.14	11	541.20	1154.14	11	135.00
rc201	1443.13	4	322.21	1433.57	4	474.00	1443.13	4	84.00
rc202	1406.18	3	274.24	1403.67	3	848.40	1407.05	3	180.60
rc203	1054.91	3	327.32	1054.91	3	1264.20	1054.91	3	319.80
rc204	886.57	3	447.36	884.75	3	1237.20	884.75	3	256.80
rc205	1256.54	3	515.78	1238.46	3	962.40	1255.84	3	143.40
rc206	1184.61	3	371.69	1197.60	3	639.60	1184.61	3	100.80
rc207	990.59	3	407.76	978.30	3	892.20	992.16	3	134.40
rc208	838.86	3	524.60	833.12	3	1014.00	833.12	3	161.40

Abbreviations hold as follows: σ - objective value, K - number of vehicles used, τ [s] - runtime in seconds.

Table 2 Detailed results of our benchmark against Schiffer et al. (2017) and Erdelić and Carić (2022).

References

Accorsi L, Vigo D (2021) A fast and scalable heuristic for the solution of large-scale capacitated vehicle routing problems. *Transportation Science* 55(4):832–856.

Beek O, Raa B, Dullaert W, Vigo D (2018) An Efficient Implementation of a Static Move Descriptor-based Local Search Heuristic. *Computers & Operations Research* 94:1–10, ISSN 0305-0548.

Christiaens J, Vanden Berghe G (2020) Slack Induction by String Removals for Vehicle Routing Problems. *Transportation Science* 54(2):417–433, ISSN 0041-1655.

Erdelić T, Carić T (2022) Goods delivery with electric vehicles: Electric vehicle routing optimization with time windows and partial or full recharge. *Energies* 15(1):285.

Froger A, Jabali O, Mendoza JE, Laporte G (2022) The Electric Vehicle Routing Problem with Capacitated Charging Stations. *Transportation Science* 56(2):460–482, ISSN 0041-1655.

Goeke D, Schneider M (2015) Routing a mixed fleet of electric and conventional vehicles. *European Journal of Operational Research* 245(1):81–99.

Groer C (2009) Vrph. `https://github.com/coin-or/VRPH`.

Irnich S (2008) A Unified Modeling and Solution Framework for Vehicle Routing and Local Search-Based Metaheuristics. *INFORMS Journal on Computing* 20(2):270–287, ISSN 1091-9856.

Kindervater GAP, Savelsbergh MWP (2003) 10. vehicle routing: handling edge exchanges. *Local Search in Combinatorial Optimization*, 337–360 (Princeton University Press).

Kullman ND, Froger A, Mendoza JE, Goodson JC (2021) frvcpy: An open-source solver for the fixed route vehicle charging problem. *INFORMS Journal on Computing* .

Mendoza J, Guéret C, Hoskins M, Lobit H, Pillac V, Vidal T, Vigo D (2014) Vrp-rep: a vehicle routing community repository.

Montoya A, Guéret C, Mendoza JE, Villegas JG (2017) The electric vehicle routing problem with nonlinear charging function. *Transportation Research Part B: Methodological* 103:87–110, ISSN 0191-2615.

Peng L, Murray C (2022) VeRoViz: A vehicle routing visualization toolkit. *INFORMS Journal on Computing* 34(4):1842–1848.

Pisinger D, Ropke S (2007) A general heuristic for vehicle routing problems. *Computers & Operations Research* 34(8):2403–2435.

Rasku J, Kwon C (2023) VeRyPy. `https://github.com/yorak/VeRyPy`.

Ropke S, Pisinger D (2006) An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science* 40(4):455–472.

Schiffer M, Klein P, Schneider M, Walther G (2017) A solution framework for a class of vehicle routing problems with intermediate stops. *Working Paper OM-DPO 01/2017* .

Schiffer M, Schneider M, Walther G, Laporte G (2019) Vehicle routing and location routing with intermediate stops: A review. *Transportation Science* 53(2):319–343.

Schiffer M, Walther G (2018) An adaptive large neighborhood search for the location-routing problem with intra-route facilities. *Transportation Science* 52(2):331–352.

Schneider M, Stenger A, Goeke D (2014) The electric vehicle-routing problem with time windows and recharging stations. *Transportation Science* 48(4):500–520.

Shaw P (1997) A new local search algorithm providing high quality solutions to vehicle routing problems. *APES Group, Dept of Computer Science, University of Strathclyde, Glasgow, Scotland, UK* 46.

Vidal T (2022) Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Computers & Operations Research* 140:105643, ISSN 0305-0548.

Vidal T, Crainic TG, Gendreau M, Prins C (2014) A unified solution framework for multi-attribute vehicle routing problems. *European Journal of Operational Research* 234(3):658–673, ISSN 0377-2217.

Zachariadis EE, Kiranoudis CT (2010) A strategy for reducing the computational complexity of local search-based methods for the vehicle routing problem. *Computers & Operations Research* 37(12):2089–2105, ISSN 0305-0548.